

Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
Department of Mechanical Engineering

Subject:	Data Science, Artificial Intelligence & Machine Learning	Total Marks:	50
Exam Date:	11th & 12th May 2026	Exam Type:	SPPU Practical Examination

Dataset used : Iris Dataset (built-in from sklearn.datasets) — No download required. Run all cells top-to-bottom in Google Colab.

Q1 — Data Acquisition & Understanding [10 Marks]

Tasks :

- Load dataset using pandas
- Display first 5 rows (df.head())
- Display last 5 rows (df.tail())
- Write 2 observations about data

Complete Code :

```
# =====
# Q1: Data Acquisition & Understanding
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
df['species'] = df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})

print("=" * 55)
print("      DATASET LOADED SUCCESSFULLY")
print("=" * 55)

# Display first 5 rows
print("\n--- First 5 Rows (head) ---")
print(df.head())

# Display last 5 rows
print("\n--- Last 5 Rows (tail) ---")
print(df.tail())

# Basic info
print("\n--- Dataset Info ---")
print(df.info())

print("\n--- Basic Statistics ---")
print(df.describe())

print("\n--- Shape of Dataset ---")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")

print("\n--- Missing Values ---")
print(df.isnull().sum())
```

Expected Output (write in practical file) :

```
=====
DATASET LOADED SUCCESSFULLY
=====
--- First 5 Rows ---
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target  species
0                5.1                3.5                1.4                0.2        0    setosa
1                4.9                3.0                1.4                0.2        0    setosa
...
--- Shape: (150, 6)      Missing Values: 0
```

Observations :

1. The Iris dataset contains **150 rows and 6 columns** with **no missing values**, making it perfectly clean and directly suitable for Machine Learning.
2. Petal length ranges from 1.0 to 6.9 cm and petal width from 0.1 to 2.5 cm — this wide spread suggests these two features can effectively distinguish between species.

Q2 — Data Visualization [10 Marks]

Tasks :

- Plot Graph 1 — Scatter Plot (Petal Length vs Petal Width)
- Plot Graph 2 — Histogram (Sepal Length distribution by species)
- Explain 2–3 line observations for each graph

Complete Code :

```
# =====
# Q2: Data Visualization
# =====
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Iris Dataset - Data Visualization", fontsize=16, fontweight='bold')

colors_map = {'setosa':'red', 'versicolor':'blue', 'virginica':'green'}

# --- Graph 1: Scatter Plot ---
for species, group in df.groupby('species'):
    axes[0].scatter(group['petal length (cm)'],
                    group['petal width (cm)'],
                    label=species, color=colors_map[species],
                    alpha=0.7, edgecolors='k', s=60)
axes[0].set_title("Scatter: Petal Length vs Petal Width")
axes[0].set_xlabel("Petal Length (cm)")
axes[0].set_ylabel("Petal Width (cm)")
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.5)

# --- Graph 2: Histogram ---
for species, group in df.groupby('species'):
    axes[1].hist(group['sepal length (cm)'], bins=15, alpha=0.6,
                 label=species, color=colors_map[species], edgecolor='black')
axes[1].set_title("Histogram: Sepal Length by Species")
axes[1].set_xlabel("Sepal Length (cm)")
axes[1].set_ylabel("Frequency")
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
print("Graphs displayed successfully!")
```

Observations to write in practical file :

Scatter Plot : Setosa (red) is clearly separated from Versicolor and Virginica based on petal dimensions. Versicolor and Virginica overlap slightly, but are still distinguishable. This confirms that petal length and width are strong discriminating features.

Histogram : Setosa has the lowest sepal length (mostly 4.5–5.5 cm), while Virginica has the highest (mostly 6.0–8.0 cm). The distributions show clear species-wise differences, confirming sepal length is also a useful feature for classification.

Q3 — Feature Preparation [10 Marks]

Tasks :

- Identify input features (X) and output label (y)
- Perform train-test split (80 % train, 20 % test)
- Explain why splitting is required

Complete Code :

```
# =====
# Q3: Feature Preparation
# =====

# Identify Input (X) and Output (y)
X = df[['sepal length (cm)', 'sepal width (cm)',
        'petal length (cm)', 'petal width (cm)']] # Input Features
y = df['target'] # Output: 0=setosa, 1=versicolor, 2=virginica

print("Input Features (X) - first 5 rows:")
print(X.head())
print("\nOutput (y) - first 5 values:")
print(y.head())
print(f"\nShape of X : {X.shape}")
print(f"Shape of y : {y.shape}")

# Train-Test Split (80 % train, 20 % test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("\n--- Train-Test Split (80-20) ---")
print(f"Training samples : {X_train.shape[0]} (80 %)")
print(f"Testing samples : {X_test.shape[0]} (20 %)")
```

Expected Output :

```
Training samples : 120 (80 %)
Testing samples : 30 (20 %)
```

Why is Train-Test Split Required ? (Write this answer)

Train-test split divides the dataset into two parts: **80 % for training** the model and **20 % for testing** its performance on completely unseen data. Without this split, we cannot know whether the model has actually learned patterns or just memorized the training data (a problem called *overfitting*). The test set acts as a real-world simulation that gives an unbiased measure of model performance.

Q4 — Option A : Classification — Random Forest [15 Marks]

Tasks :

- Apply Random Forest Classifier
- Train model and predict on test data
- Calculate Accuracy, print Classification Report & Confusion Matrix

Complete Code :

```
# =====
# Q4 - Option A: Classification using Random Forest
# =====
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)

# Step 1: Create model
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Step 2: Train model
model.fit(X_train, y_train)
print("Model Training Complete!")

# Step 3: Predict on test data
y_pred = model.predict(X_test)

print("\n--- Predictions vs Actual (first 10) ---")
results = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print(results.head(10))

# Step 4: Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\n■ Model Accuracy : {accuracy * 100:.2f} %")

print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred,
                             target_names=['Setosa', 'Versicolor', 'Virginica']))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Setosa', 'Versicolor', 'Virginica'],
            yticklabels=['Setosa', 'Versicolor', 'Virginica'])
plt.title("Confusion Matrix - Random Forest Classifier")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.show()
```

Expected Output :

```
Model Training Complete!
■ Model Accuracy : 100.00 %

--- Classification Report ---
              precision    recall  f1-score   support
Setosa                1.00      1.00       1.00        10
Versicolor            1.00      1.00       1.00         9
Virginica              1.00      1.00       1.00        11
accuracy                           1.00         30
```

Note : A Confusion Matrix heatmap will also be displayed as a graph in Colab.

Q4 — Option B : Regression — Linear Regression [15 Marks]

(Choose EITHER Option A OR Option B — not both. Option A recommended for higher accuracy.)

Tasks :

- Apply Linear Regression (predict Petal Length from Sepal features)
- Train model and predict output
- Calculate error metrics: R^2 , MSE, RMSE, MAE

Complete Code :

```
# =====
# Q4 - Option B: Linear Regression
# =====
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Use sepal features to predict petal length
X_reg = df[['sepal length (cm)', 'sepal width (cm)']]
y_reg = df['petal length (cm)']

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42)

# Create & train model
reg_model = LinearRegression()
reg_model.fit(X_train_r, y_train_r)
print("Linear Regression Model Trained!")

# Predict
y_pred_r = reg_model.predict(X_test_r)

print("\n--- Predicted vs Actual (first 10) ---")
comp = pd.DataFrame({'Actual': y_test_r.values, 'Predicted': y_pred_r.round(2)})
print(comp.head(10))

# Error metrics
r2 = r2_score(y_test_r, y_pred_r)
mse = mean_squared_error(y_test_r, y_pred_r)
mae = mean_absolute_error(y_test_r, y_pred_r)
rmse = np.sqrt(mse)

print(f"\n■ R² Score : {r2:.4f} (closer to 1 = better)")
print(f"■ MSE : {mse:.4f} (lower = better)")
print(f"■ RMSE : {rmse:.4f}")
print(f"■ MAE : {mae:.4f}")

# Actual vs Predicted plot
plt.figure(figsize=(7, 5))
plt.scatter(y_test_r, y_pred_r, color='blue', alpha=0.7, edgecolors='k')
plt.plot([y_test_r.min(), y_test_r.max()],
         [y_test_r.min(), y_test_r.max()], 'r--', lw=2, label='Ideal Fit')
plt.xlabel("Actual Petal Length"); plt.ylabel("Predicted Petal Length")
plt.title("Linear Regression: Actual vs Predicted")
plt.legend(); plt.grid(True, alpha=0.4); plt.show()
```

Expected Output :

```
Linear Regression Model Trained!
■ R² Score : 0.7694 (closer to 1 = better)
■ MSE : 0.4011 (lower = better)
■ RMSE : 0.6334
■ MAE : 0.4878
```

What these metrics mean :

Metric	Formula / Meaning	Good Value
R ² Score	Proportion of variance explained by the model	Close to 1.0
MSE	Mean of squared differences (Actual – Predicted) ²	As low as possible
RMSE	Square root of MSE — same unit as target variable	As low as possible
MAE	Mean of absolute differences Actual – Predicted	As low as possible

Q5 — Conclusions & Viva [5 Marks]

Write these 4 conclusions in your practical file :

Conclusion 1 — Dataset :

The Iris dataset was successfully loaded using pandas from sklearn. It contains 150 records, 4 numerical features, and 3 target classes (Setosa, Versicolor, Virginica) with no missing values, making it an ideal dataset for ML experiments.

Conclusion 2 — Visualization :

Data visualization using scatter plots and histograms revealed that petal length and petal width are the most discriminating features. Setosa is clearly separable from the other two species even in a 2D scatter plot.

Conclusion 3 — Classification (Option A) :

The Random Forest Classifier achieved 100 % accuracy on the 30-sample test set, proving it effectively learned the species boundaries. The confusion matrix confirmed zero misclassifications, which shows excellent generalization.

Conclusion 4 — Splitting & Generalization :

The 80-20 train-test split ensured an unbiased evaluation of the model. The high accuracy on completely unseen test data confirms the model has generalized well without overfitting to the training data.

Important Viva Questions & Answers

Q1. What is a dataset?

Ans: A dataset is a structured collection of data (rows = samples, columns = features) used to train and evaluate Machine Learning models.

Q2. What is the Iris dataset?

Ans: The Iris dataset is a classic ML dataset with 150 samples of iris flowers, 4 features (sepal/petal length & width), and 3 species labels.

Q3. Why do we perform train-test split?

Ans: To evaluate model performance on unseen data and detect overfitting. We train on 80 % and test on the remaining 20 %.

Q4. What is accuracy?

Ans: $\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions}) \times 100$. It measures how often the classifier is correct.

Q5. What is R² Score?

Ans: R² (R-squared) measures what proportion of variance in the output is explained by the model. R² = 1 means perfect prediction; R² = 0 means no predictive power.

Q6. What is MSE?

Ans: Mean Squared Error = average of squared differences between actual and predicted values. Lower MSE = better model. Sensitive to large errors.

Q7. What is overfitting?

Ans: Overfitting occurs when the model memorizes training data and performs poorly on new/test data. It means the model has not generalized well.

Q8. What is a Random Forest?

Ans: Random Forest is an ensemble ML algorithm that builds many Decision Trees and combines their predictions (majority vote) for better accuracy and stability.

Q9. What is Linear Regression used for?

Ans: Linear Regression predicts a continuous numeric output by fitting a straight line ($y = mx + c$) through the training data.

Q10. What is pandas used for?

Ans: Pandas is a Python library for data manipulation and analysis. It provides DataFrame structures to load, clean, explore and transform datasets.

HOW TO RUN IN GOOGLE COLAB : Go to colab.research.google.com → New Notebook → Copy each question's code block into a separate cell → Press **Shift + Enter** to run. All libraries (pandas, sklearn, matplotlib, seaborn) are pre-installed in Colab. No dataset download is needed — the Iris dataset loads automatically from sklearn.